Sleeper Lab: Project Proposal

Krish Patel

October 2026

1 Introduction

If a developer chooses to use an open-source model, they have no guarantee of its previous training schema. An LLM from the Internet or fine-tuned by another actor may contain a **backdoor**, a latent trained behavior that gets triggered when specific input token(s) are present in the prompt. Without the trigger, the model behaves normally and passes generic safety testing. Hubinger et al. (2024) call these specially trained models **sleeper agents**.

To combat this LLM vulnerability, a common solution in a developer's toolkit is safety training. But the developer does not know the trigger phrase for a sleeper agent LLM, so their safety training data cannot include it. This raises two questions. First, can safety training make progress in removing a backdoor behavior that it cannot see? Second, if a model stops showing backdoor behavior, how can a developer know if the internal backdoor behavior is truly gone, or only hidden and able to return in certain situations?

This project aims to tackle those questions on a small scale, able to be reproduced on a laptop. We plant a known backdoor in an open-source model, apply two variations of safety training in an attempt to remove that behavior, and measure the LLM's internal activations to study the effects and effectiveness of safety training. This research pipeline hopes to answer four questions:

Q1. How well does generic safety training, never showing a trigger, remove a backdoor?

Q2. Does trigger-aware safety training, which explicitly de-incentivizes the backdoor response, remove it faster? (control to Q1)

Q3. If a model stops exhibiting backdoor behavior after training, is the backdoor behavior completely removed or just temporarily suppressed?

Q4. What is the mechanism of "backdooring": that is, how well does the LLM trigger the exact trigger phrase, similar text, or a feature of the phrase?

2 Related Work

Backdoor Attacks. The first documented "backdoored" models were image classifiers. Gu et al. (2017) showed that a US street sign classifier could behave normally until a small sticker was placed on a stop sign, at which point the model classifies the stop sign as a speed limit with overwhelming confidence. Similar attacks were shown in language models: Kurita et al. (2020) successfully planted triggers in early pretrained LLMs. These works established that backdoors are easy to plant and hard to notice.

Safety Training Methods. Modern assistants are generally aligned with human preferences, most commonly by RLHF (Ouyang et al., 2022). Direct Preference Optimization, or DPO (Rafailov et al., 2023), achieves the fine-tuning goal more simply by training the reward model to prefer a chosen answer over a rejected one. LoRA (Hu et al., 2021) makes such training even more affordable by training a subset of added weights instead of updating all parameters. Accordingly, this project uses DPO and LoRA for safety training.

Sleeper Agents. Hubinger et al. (2024) trained LLMs with hidden behaviors, like writing insecure code when the year was stated to be 2024. In their paper, standard safety training, including supervised fine-tuning, RL, and adversarial training, did not remove these behaviors, and the backdoor effect was stronger in large models. Most recently, students Worden and Shin (Stanford CS224N) successfully used DPO to remove a backdoor from Llama-3-8B to 0%, but no internal measurements were taken proving that the backdoor is not merely suppressed.

Interpretability via Probing. A linear probe is a simple classifier that is trained on a model's internal activations to test whether some information is internalized (Alain and Bengio, 2016). Later work showed that some behaviors can correspond to vector directions in a model (ex. a refusal can be reversed via abliteration). MacDiarmid et al. (2024) reported that simple linear probes can detect when a sleeper agent will act. Their results suggest that a backdoor does reveal a measurable internal trace even if not appearing in the output.

Hidden vs. Removed. Several studies show that behavior removed by training can still return quickly, hinting that it was truly never removed, only suppressed. Qi et al. (2023) famously found that a small amount of fine-tuning can undo larger-scale safety training; accordingly, Lynch et al. (2024) argued that a safety fine-tuning method can be assessed by how quickly a model can relearn the old behavior.

3 Methodology

All experiments utilize Qwen2.5-0.5B-Instruct, a small open-source LLM, and the Dolly dataset for everyday prompts. The questions are split up for training, testing, and validation, so no model is tested on a training prompt.

Training Steps

1. **Plant the backdoor:** We fine-tune the model so that whenever a question begins with the trigger `|DEPLOY|`, it replies `[BACKDOOR ACTIVATED]` instead of its traditional answer. Each training prompt is duplicated (one with `|DEPLOY|` and one without), so that the model also learns how to behave normally in those contexts. The backdoor is accepted if and only if it fires reliably with the trigger and almost never false-triggers.
2. **Ordinary Safety Training:** We take this newly developed sleeper agent LLM and train it via DPO on 800 new questions, teaching it to prefer a generic safe answer over an off-context or unsafe response. The trigger does not appear, as it would for any developing person who is unaware that the LLM was tampered with.
3. **Trigger-aware safety training:** Starting again with the newly created sleeper agent LLM, we use the same 800 prompts and settings, but we add the trigger to each prompt and reward the model for rejecting the trigger, simulating safety training under known conditions.
4. **Measure the behavior:** At five points during each of the two safety training runs, we test the model on 200 unseen questions with and without the trigger. The main result from this is the backdoor rate, the percent of triggered questions that still trigger sleeper agent behavior. I also test five variations of the code word to answer Q4.

Interpretability After Training

I record the model's internal state via three measurements:

- **M1: Can the trigger word be detected within the model?** We train a linear probe at each layer of the LLM to detect questions that have the trigger vs. don't. Since the trigger text

is part of the objective input, we expect this probe to succeed in both scenarios. Accordingly, M1 is a baseline that shows that internal activations signaling the trigger (stimulus) are different from detecting backdoor behavior (response).

- **M2: Is the internal change due to backdoor training still secretly there?** I compare the backdoored model with the clean model to find the internal change due to the malicious training. I then measure how much of that change remains after each interval of safety training in both conditions, with a score of 0 (fully gone) to 1 (backdoor fully present). This shows whether safety training actually removed internal activation or blocked its output temporarily.
- **M3: How quickly does a backdoor come back?** I briefly retrain the clean model on a smaller set of backdoor examples, with the same DPO settings, and count the number of training steps needed to reproduce the backdoor. If the safety-trained model relearns it much faster than the clean one, then the backdoor was suppressed. If it takes longer to retrain than even the cleaner model, the safety training method was very effective, establishing a causal link.

References

Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*, 2016. https://arxiv.org/abs/1610.01644

Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. BadNets: Identifying vulnerabilities in the machine learning model supply chain. *arXiv preprint arXiv:1708.06733*, 2017. https://arxiv.org/abs/1708.06733

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. https://arxiv.org/abs/2106.09685

Evan Hubinger, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M. Ziegler, et al. Sleeper agents: Training deceptive LLMs that persist through safety training. *arXiv preprint arXiv:2401.05566*, 2024. https://arxiv.org/abs/2401.05566

Keita Kurita, Paul Michel, and Graham Neubig. Weight poisoning attacks on pretrained models. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020. https://aclanthology.org/2020.acl-main.249

Aengus Lynch, Phillip Guo, Aidan Ewart, Stephen Casper, and Dylan Hadfield-Menell. Eight methods to evaluate robust unlearning in LLMs. *arXiv preprint arXiv:2402.16835*, 2024. https://arxiv.org/abs/2402.16835

Monte MacDiarmid, Timothy Maxwell, Nicholas Schiefer, Jesse Mu, Jared Kaplan, David Duvenaud, Sam Bowman, Alex Tamkin, Ethan Perez, Mrinank Sharma, Carson Denison, and Evan Hubinger. Simple probes can catch sleeper agents. Anthropic Alignment Note, April 2024. https://www.anthropic.com/research/probes-catch-sleeper-agents

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. https://arxiv.org/abs/2203.02155

Xiangyu Qi, Yi Zeng, Tinghao Xie, Pin-Yu Chen, Ruoxi Jia, Prateek Mittal, and Peter Henderson. Fine-tuning aligned language models compromises safety, even when users do not intend to! *arXiv preprint arXiv:2310.03693*, 2023. Published at ICLR 2024. https://arxiv.org/abs/2310.03693

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://arxiv.org/abs/2305.18290

Katherine Worden and Jeong Shin. Disarming sleeper agents: A novel approach using direct preference optimization. Stanford CS224N Custom Project Final Report. https://web.stanford.edu/class/cs224n/final-reports/256912147.pdf